PSK from the examples in section 12.2.1. COSE requires a `Recipient` object to be created for each recipient of a message and the HKDF algorithm is specified at this level. This allows different key derivation or wrapping algorithms to be used for different recipients of the same message, but in this example there's only a single recipient. The algorithm is specified by adding an attribute to the recipient object. You should add these attributes to the `PROTECTED` header region, to ensure they are authenticated. The random nonce is also added to the recipient object, as the `HKDF_Context_PartyU_nonce` attribute; I'll explain the `PartyU` part shortly. You then create an `EncryptMessage` object and set some content for the message. Here I've used a simple string, but you can also pass any array of bytes. Finally, you specify the content encryption algorithm as an attribute of the message (a variant of AES-CCM in this case) and then encrypt it.

Listing 12.13 Encrypting a message with COSE HKDF

```
Security.addProvider(new BouncyCastleProvider());    #A
var keyMaterial =
PskServer.loadPsk("changeit".toCharArray());    #B

var recipient = new Recipient();    #C
var keyData = CBORObject.NewMap()    #C
        .Add(KeyKeys.KeyType.AsCBOR(),
KeyKeys.KeyType_Octet)    #C
        .Add(KeyKeys.Octet_K.AsCBOR(), keyMaterial);    #C
recipient.SetKey(new OneKey(keyData));    #C
recipient.addAttribute(HeaderKeys.Algorithm,    #D
        AlgorithmID.HKDF_HMAC_SHA_256.AsCBOR(),    #D
        Attribute.PROTECTED);    #D
var nonce = new byte[16];    #E
new SecureRandom().nextBytes(nonce);    #E
```